SuperHybrid: Single-Pass Black-Box Uncertainty Quantification for LLM Reasoning Traces

Youxuan Ma
youxuan.ma@yale.edu
Peng Chen
peng.chen.pc838@yale.edu
Lixing Lin
lixing.lin@yale.edu

Abstract

We ask whether a *single* reasoning trace from a reasoning LLM, observed only as black-box surface text, contains enough signal to predict whether the model's final answer is correct—at one generation, with no logits, no sampling, and no model-side self-report. We introduce **SuperHybrid**, a stacking pipeline that combines four complementary views of a trace: (i) a fine-tuned text encoder over the trace tail, (ii) a problem-conditioned cross-encoder, (iii) a multi-layer probe over the generator's hidden states at the answer-marker position, and (iv) a small set of content-free behavioral features parsed from the trace text. Across 6 378 traces spanning four benchmarks (MATH500, GSM8K, GPQA-Diamond, ARC-Challenge) and two reasoning-distilled models (DeepSeek-R1-Distill-Qwen-7B and Llama-8B), SuperHybrid reaches pooled AUROC **0.815** (LR meta-learner) / **0.807** (RF) with expected calibration error **0.042**, beating a fine-tuned DeBERTa-v3 read of the trace by $+0.05$ **AUROC** (paired DeLong $p < 10^{-19}$) and halving its ECE. On MATH500-Qwen7B alone the system reaches **0.929** AUROC. An ablation across 328 additional handcrafted structural features (n-gram motifs, graph topology, timing, persistent homology) adds ≤ 0.005 AUROC—a *text-saturation* result that informs the recipe: a small text + probe + random-forest stack is enough.

1 Introduction

When an LLM solves $x^2 + 5x + 6 = 0$ and returns $x = -2$ or $x = -3$, a user able to verify the work would not have needed the LLM in the first place. This *verifiability gap* is the core deployment problem for reasoning-style LLMs, and three established families of uncertainty quantification (UQ) all fail to close it cheaply. **Logit-based methods** (perplexity, token entropy) need white-box access to the generator—increasingly unavailable as frontier models go API-only. **Sampling methods** (self-consistency, semantic entropy) require $N \geq 5$ independent generations per query, multiplying inference cost 5–20×—prohibitive when reasoning traces already span thousands of tokens. **Verbalized confidence** ("How confident are you, 0–100?") is empirically miscalibrated: RLHF rewards confident guessing, and frontier models are systematically over-confident.

A reasoning trace, however, already contains structural information that humans use intuitively to spot uncertain reasoning—backtracking ("wait, that's wrong"), verification ("let me check"), restarts, and rumination. Marjanović et al. [1] document on AIME-24 that *correct* DeepSeek-R1 traces average $\approx 2{,}000$ tokens whereas *incorrect* ones average $\approx 4{,}000$, and rumination rate is negatively correlated with accuracy. Our claim is that a single such trace, read as surface text plus the generator's intermediate hidden states at one position, can drive a calibrated UQ pipeline at one-fold compute.

Contributions. (i) We propose **SuperHybrid**, a four-signal stacked predictor of single-trace correctness that operates at one generation, requires only the trace text and a single hidden-state slice,

Preprint.

and matches or beats sampling-based UQ at a fraction of the cost. (ii) We report pooled AUROC 0.815 with ECE 0.042 across 4 benchmarks $\times$ 2 models, with a peak of 0.929 on MATH500-Qwen7B and a $+0.05$ AUROC lift over a fine-tuned DeBERTa-v3 baseline. (iii) We provide an ablation showing that 328 handcrafted structural features (n-gram motifs, graph topology, inter-event timing, persistent homology) add ≤ 0.005 AUROC once strong text encoders and a multi-layer probe are present—a *text-saturation* result that informs how minimal a useful structural-UQ stack can be. (iv) We demonstrate cross-domain transfer (MATH500 $\rightarrow$ GPQA-Diamond) with AUROC 0.73, showing that the signal generalizes beyond the training domain when trace shapes are similar.

2 Related Work

Topology of reasoning. Minegishi et al. [2] cluster the hidden activations of a reasoning model and study the resulting reasoning *graph*—cyclicity, diameter, small-world index. Their work requires white-box access and treats topology as an interpretability lens rather than a UQ predictor; we instead operate on text-surface traces and predict per-instance correctness. Persistent-homology work on traces and Da et al. [3] target *explanation* or quality assessment rather than answer correctness. Gandhi et al. [4] define the six-way cognitive behavior taxonomy (forward, verify, revise, restart, hesitate, conclude) we adopt; [1] provides the empirical link between behavior frequencies and accuracy.

Single-pass black-box UQ. Verbalized confidence [5] and trace-length heuristics are known to miscalibrate. Sampling baselines—self-consistency [6] and semantic entropy [7]—require $N \geq 5$–8 generations per query; LM-Polygraph [8] consolidates these for benchmarking. Our work extends single-trace UQ by combining content-aware text encoders, a problem-conditioned cross-encoder, and an internal-state probe in one stacked predictor. Probing intermediate transformer activations as a UQ predictor is motivated by the empirical observation that mid-late layers hold the most generalization-relevant features.

3 Method

3.1 Four complementary signals

SuperHybrid combines four views of a trace, each producing a calibrated probability or a small feature vector that becomes input to a stacking meta-learner.

(i) Text surface, P_{text}. A fine-tuned DeBERTa-v3-base classifier on the last 512 tokens of the trace ("trace tail"), 5-fold stratified cross-validation. The trace tail concentrates the model's final consolidation step.

(ii) Problem alignment, P_{cond}. A problem-conditioned cross-encoder DeBERTa reading (problem $\|$ trace tail). Captures topic drift—when the trace becomes inconsistent with the original question.

(iii) Internal state, P_{probe}. A small MLP probe over the generator's hidden states, concatenated across *five* decoder layers spread across the trunk (relative depths $\approx 14, 29, 43, 57, 71\%$) and *two* token positions (the answer-marker and the last-token of the trace). Empirically the mid-late layers (≈ 57–72% depth across the two backbones) carry the cleanest single-layer signal, consistent with the probing literature.

(iv) Behavioral features, F. A 28-dim vector parsed using the six-class behavior taxonomy (forward / verify / revise / restart / hesitate / conclude): length and proportions, structural features (verification-to-forward ratio, backtrack position, transition entropy, behavior cycle count, longest forward run), and content-free meta-features (wait-density, question-mark density, repetition rate).

3.2 Stacking with leakage-safe out-of-fold predictions

Each base predictor is trained with 5-fold stratified cross-validation and emits *out-of-fold* (OOF) probabilities—for every item, the prediction comes from a fold in which that item was held out. The meta-learner consumes only OOFs as features and is itself wrapped in a fresh outer 5-fold. Each item's stacked prediction is therefore held out *twice*, eliminating the multi-seed re-aggregation inflation common in stacking pipelines.

We compare three meta-learners (logistic regression, random forest, XGBoost). Random forest is reported as the default operational classifier because it consistently halves the calibration error at indistinguishable AUROC.

Evaluation

We report AUROC, AUPRC, expected calibration error (ECE; 10 uniform bins), and accuracy at fixed coverage (selective generation). Statistical tests use DeLong 95% confidence intervals (logit-transformed) on every pooled AUROC and *paired* DeLong tests on every challenger-vs-baseline pair, fanned out across overall, model-family, dataset-family, and per-cell slices.

4 Experimental Setup

Datasets and generation. We use four publicly released test splits: MATH500 [10] (a 500-item subset of the MATH competition-mathematics benchmark), GSM8K [11] (1,319 grade-school word problems), GPQA-Diamond [12] (198 graduate-level science MCQ), and ARC-Challenge [13] (1,172 science MCQ). Each item is solved by both DeepSeek-R1-Distill-Qwen-7B and Llama-8B [9] (open-weight, MIT and Llama Community licenses) at $T = 0.6$, top-$p = 1$, max 32,768 tokens, exactly one sample per prompt — 6,378 traces across 8 dataset×model cells. Final answers are extracted from outside the `<think>` block and scored by exact-match (math) or letter-match (MCQ); no other preprocessing is applied to the trace text.

Behavior parser. Sentences are segmented with a math-aware spaCy pipeline and labeled by a priority-ordered keyword/regex matcher (revise > restart > verify > conclude > hesitate > forward). Inter-annotator agreement against 100 manually-labeled traces is Cohen's $\kappa = 0.74$.

Compute and baselines. Trace generation, hidden-state extraction, and DeBERTa fine-tuning ran on Yale University's Grace HPC cluster (NVIDIA A100/H100 GPUs allocated via SLURM, PyTorch 2.2); behavioral-feature parsing, stacking, and statistical analysis are CPU-only on a workstation. Hyperparameters and the Optuna tuning protocol are detailed in Appendix B. We compare SuperHybrid against (a) length-only logistic regression; (b) seven lexical surface cues; (c) the 28 handcrafted behavioral features alone; (d) TF-IDF unigram+bigram logistic regression; (e) plain DeBERTa-v3-base fine-tuned on the trace tail—the text-encoder bar a structural-UQ method must clear to claim that structure adds value over content.

5 Results

5.1 Pooled AUROC and calibration

Table 1 reports pooled AUROC and ECE across the eight cells ($n = 6{,}344$–$6{,}378$ depending on OOF coverage). 95% DeLong confidence intervals are shown for the most relevant comparators.

Table 1: Pooled AUROC and ECE across 8 dataset×model cells.

Method	AUROC	95% CI	ECE
Length-only LR	0.595	—	—
Lexical cues LR (7 feats)	0.628	—	—
Behavioral features alone (28)	0.666	[0.652, 0.679]	—
TF-IDF LR (≈20k feats)	0.707	—	—
Plain DeBERTa-v3, trace tail	0.762	[0.751, 0.774]	0.090
Problem-conditioned DeBERTa	0.788	[0.777, 0.799]	0.086
Multi-layer probe (alone)	0.776	[0.764, 0.787]	—
Mean of three probabilities	0.805	[0.792, 0.813]	0.082
SuperHybrid (LR meta-learner)	**0.815**	**[0.804, 0.826]**	0.080
SuperHybrid (RF meta-learner)	**0.807**	**[0.797, 0.818]**	**0.042**

SuperHybrid beats the plain DeBERTa text-encoder bar by $+0.045$ (RF) to $+0.053$ (LR) AUROC, paired DeLong $p < 10^{-19}$. The RF variant *also* halves ECE from 0.090 to 0.042 at indistinguishable AUROC—random forest's tree-mean output is doing the calibration work, and we recommend RF as the default operational meta-learner.

5.2 Per-cell results and cross-domain transfer

Table 2 (left) shows AUROC of SuperHybrid (RF) per cell, strongest on math; Qwen cells are uniformly 0.05–0.10 above Llama, an asymmetry that persists across every base predictor. The right panel reports out-of-domain AUROC when we train RF on MATH500 features and evaluate without retraining: MATH500 $\rightarrow$ GPQA-Diamond transfers at 0.73, *above* the length-only baseline (0.65)—the signal is genuinely structural, not length-driven. Transfer fails to ARC and GSM8K (≈ 0.51–0.58), whose trace shapes (short MCQ rationales, grade-school arithmetic chains) differ markedly from MATH500's competition-mathematics traces.

Table 2: (Left) per-cell AUROC, SuperHybrid (RF). (Right) out-of-domain AUROC.

Dataset	Qwen-7B	Llama-8B
MATH500	**0.929**	0.869
GSM8K	0.859	0.763
GPQA-Diamond	0.780	0.683
ARC-Challenge	0.731	0.683

Target	n	RF	Length-only
GPQA-Diamond	198	**0.732**	0.645
ARC-Challenge	1172	0.577	0.584
GSM8K	1319	0.508	0.492

5.3 What does and does not contribute: text-saturation

We expanded the structural feature set from 28 handcrafted features to 328 features by adding behavior n-gram motifs (231 features), trace graph topology (15), inter-event timing (46), and content-free persistent-homology descriptors (36). Stacking the 328-feature block on top of the four signals in Section 3.1 yields the leave-one-family-out deltas in Table 3.

Table 3: Ablation. ΔAUROC of removing each family from the all-in stack (positive = contributes, negative = hurts).

Removed family	ΔLR	ΔRF	ΔXGB
Text encoders (DeBERTa, conditioned, RoBERTa)	$+0.036$	$+0.043$	$+0.036$
Multi-layer probe	$+0.010$	$+0.012$	$+0.011$
328 expanded structural features	-0.021	-0.005	$+0.002$

Three observations: (a) text encoders are load-bearing—removing them collapses AUROC by 0.04, four times any other family; (b) the multi-layer probe contributes a uniform $+0.01$, the only consistent non-text gain; (c) expanding the behavioral block from 28 to 328 features adds zero or negative value once text encoders and the probe are present (LR is *hurt* by -0.021). We interpret (c) as **text-saturation**: the lexical signatures of revision ("wait, that's wrong"), verification ("let me check"), and restart ("starting over") that drive structural-feature counts also feed the text encoder directly. Handcrafted features remain useful for *interpretability* and pass a length-control test (they beat length-only inside every length quintile, $\Delta\text{AUROC} = 0.04$–$0.08$ per bin), but add no stacking-level signal beyond the small probe gain.

6 Discussion

Calibration and the minimal recipe. LR sits near ECE 0.080 and RF near 0.042; the AUROC distinction is within paired-DeLong noise ($\Delta \approx 0.005$), but the calibration gap is two-fold and structural—RF's tree-mean averaging halves it. Combined with text-saturation, this argues for a deliberately *minimal* stack: trace-tail encoder + problem-conditioned encoder + multi-layer probe + small behavioral vector, stacked with RF. The recipe is computationally light—one generation plus one teacher-forcing pass. At 70% coverage on MATH500, SuperHybrid answers $\approx 95\%$ of kept items correctly, competitive with self-consistency $N=8$ ($\approx 92\%$) at one-eighth the cost.

Limitations, future work, and takeaway. The system applies only to open-source models and verifiable benchmarks; a persistent Qwen $\leftrightarrow$ Llama asymmetry of ≈ 0.06 AUROC remains unexplained. Future directions include running the probe at every k-th token for mid-trace abort, distilling it into a text-only student to remove the white-box requirement, and extending to medical, legal, and code reasoning. Overall, SuperHybrid shows that calibrated single-pass UQ from one reasoning trace is achievable, and that the right recipe is a deliberately minimal text + probe + RF stack.

References

[1] S. Marjanović et al. DeepSeek-R1 Thoughtology: Let's Think About LLM Reasoning. *arXiv:2504.07128*, 2025.

[2] G. Minegishi et al. Topology of Reasoning: Understanding Large Reasoning Models through Reasoning Graph Properties. *arXiv:2506.05744*, 2025.

[3] L. Da et al. Understanding the Uncertainty of LLM Explanations: A Perspective Based on Reasoning Topology. In *KDD*, 2025.

[4] K. Gandhi et al. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars. *arXiv:2503.01307*, 2025.

[5] Z. Lin, S. Trivedi, and J. Sun. Generating with confidence: Uncertainty quantification for black-box large language models. *arXiv:2305.19187*, 2023.

[6] X. Wang et al. Self-Consistency Improves Chain-of-Thought Reasoning in Language Models. *arXiv:2203.11171*, 2022.

[7] L. Kuhn, Y. Gal, and S. Farquhar. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. *arXiv:2302.09664*, 2023.

[8] R. Vashurin et al. Benchmarking Uncertainty Quantification Methods for LLMs with LM-Polygraph. *TACL*, 2025.

[9] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025.

[10] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring Mathematical Problem Solving with the MATH Dataset. *arXiv:2103.03874.*, 2021.

[11] K. Cobbe et al. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*, 2021.

[12] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv:2311.12022*, 2023.

[13] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv:1803.05457*, 2018.

[14] P. He, J. Gao, and W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. *arXiv:2111.09543.*, 2021.

[15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A Next-Generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019.

A Code Repo

The code for SuperHybrid is available at `https://github.com/markyx316/LLM-Reasoning-Trace-Topology`

B Computing Infrastructure and Hyperparameters

Compute resources. All trace generation, hidden-state extraction, and DeBERTa fine-tuning were performed on Yale University's *Grace* HPC cluster, with jobs scheduled via SLURM on NVIDIA B200 and H200 GPUs from the `gpu_b200` and `gpu_h200` partition. Software stack: PyTorch 2.2.0, HuggingFace Transformers ≥ 4.40, sentence-transformers, spaCy 3.x, `torch_geometric`, scikit-learn 1.x, XGBoost 3.2, Optuna 4.x. Aggregate GPU usage across the project was ≈ 60 GPU-hours. All meta-learning, behavioral-feature parsing, and statistical analysis were run CPU-only.

Trace generation. Temperature $T = 0.6$, top-$p = 1.0$, no repetition penalty, max $32{,}768$ tokens, exactly one sample per prompt. Each model is loaded in bf16 for generation; the official chat template is applied with the standard system prompt. Generation is performed once per prompt; we do *not* use sampling baselines.

DeBERTa-v3 fine-tuning (signal P_{text} and P_{cond}). Backbone: `microsoft/deberta-v3-base` [14]. Input: last 512 trace tokens (for P_{text}) or `[CLS]` + problem + `[SEP]` + trace tail (for P_{cond}), truncated to 512. Optimizer AdamW ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$), weight decay 0.01 (zero on bias and LayerNorm), peak learning rate 2×10^{-5}, linear warmup over 6% of steps then linear decay, batch size 8, 3 epochs. Models are loaded in fp32 (an early bf16 fallback caused saturation at AUROC 0.5 and was patched). Training is repeated under 5-fold stratified CV with `random_state=42`; out-of-fold probabilities are saved as the OOF input to the meta-learner.

Multi-layer hidden-state probe (signal P_{probe}). Hidden states are extracted by a single teacher-forcing pass over the saved trace at five depths spread across the trunk (layers $\{4, 8, 12, 16, 20\}$ of the 28-layer Qwen-7B / hidden-dim 3584 and the proportionally matched layers of the 32-layer Llama-8B / hidden-dim 4096, i.e. relative depths $\approx 14, 29, 43, 57, 71\%$) and at two token positions (the answer-marker and the last-token of the trace); the resulting $5 \times 2 \times d$ tensor is flattened and concatenated as the probe input. Probe head: 2-layer MLP with hidden size 256, ReLU activations, dropout 0.3, sigmoid output. Optimizer AdamW, learning rate 10^{-3}, weight decay 10^{-3}, batch size 128, 25 epochs with cosine-annealing schedule. 5-fold stratified CV; `random_state=42`.

Behavioral feature pipeline (signal F). spaCy `en_core_web_sm` tokenizer with a custom math-aware sentence segmenter. Six-class taxonomy (forward / verify / revise / restart / hesitate / conclude) by priority-ordered regex matcher; 28 scalar features per trace (length and proportion, structural, content-free meta). Inter-annotator agreement Cohen's $\kappa = 0.74$ over 100 manually labeled traces.

Stacking meta-learners. All operate on the OOF probability matrix produced by the four base predictors, plus the 28-dim behavioral vector.

- *Logistic regression.* L2 penalty, class_weight = balanced, $C = 1.0$, max_iter = $2{,}000$, features standard-scaled per fold (sklearn defaults with class-imbalance correction).
- *Random forest.* 300 trees, max_depth = None, min_samples_leaf = 5, class_weight = balanced, $n\_jobs = -1$, `random_state=42`.
- *XGBoost.* $n\_estimators = 300$, max_depth 6, $\eta = 0.05$, scale_pos_weight = $n_{\text{neg}}/n_{\text{pos}}$, eval_metric = logloss, tree_method = hist, $n\_jobs = -1$, `random_state=42`. The configuration was guided by an earlier $2{,}000$-trial Optuna [15] study on the Phase-1 hybrid stacker (study log: `data/optuna_hybrid_v1_clean.db`) that explored $\eta \in [10^{-3}, 0.3]$ log-uniform, max_depth $\in \{3, \ldots, 10\}$, subsample $\in [0.5, 1.0]$, colsample_bytree $\in [0.5, 1.0]$, n_estimators $\in \{100, \ldots, 800\}$, $\gamma \in [0, 5]$, $\lambda \in [10^{-3}, 10]$, $\alpha \in [10^{-3}, 10]$. The wider Optuna search did not statistically beat the practical defaults above for the SuperHybrid stacker, so we kept the simpler hardcoded configuration.

Outer cross-validation for the stacker. Fresh 5-fold stratified split with `random_state=42`, identical fold assignment to the base learners' inner CV. Each item is therefore held out twice (once by every base learner, once by the meta-learner), making the stacked prediction leakage-safe at the item level.

Statistical inference. DeLong 95% CIs are computed in the logit space and back-transformed for asymmetry stability. Paired DeLong tests align two predictors on the composite key $(\text{group}, \text{item\_id})$, use the empirical kernel-matrix estimator for the variance of the AUROC difference, and report two-sided p-values. Tests fan out across overall, model-family, dataset-family, and per-cell slices (8 cells) without multiple-comparison correction in the body table; per-slice p-values should be read as exploratory.

C AI Use Statement

We used Anthropic's Claude (Sonnet 4.6 / Opus 4.6) as a coding and writing assistant during this project. It helped us with code review and debugging, as well as some statistical analysis. We also used it to review our report and incorporated its suggestions to polish and enhance our final draft. In addition, we used it as an idea sounding-board and discussed with it whether to drop or retain individual signal families and what ablations would be informative.

D Representative Prompts

A small selection of the prompts we issued to Claude during our research process.

1. *Statistical interpretation.* "I just got the new paired DeLong results and synced them. Please meticulously examine, think, and reason through each of them step by step in detail. Then demonstrate what they mean exactly and what the implications are."
2. *Diagnosis.* "I received this error from the trace-DAG build: [traceback]. Please diagnose and address the error step by step in detail."
3. *LaTeX porting.* "Create the LaTeX version of our report following this NeurIPS format, and make sure it's compliant with all the requirements."
4. *Model selection.* "We keep getting the same DeBERTa results. We need a genuinely new method. Search for alternatives and recommend the best option for improving AUROC on long reasoning traces."
5. *Performance diagnosis.* "Check why training is so slow—each epoch takes 24 minutes. Identify the bottleneck and fix it."
6. *Feature engineering.* "Implement focal loss, label smoothing, and early stopping in the Longformer fine-tuning loop. Add `-focal-gamma`, `-label-smoothing`, and `-patience` arguments so we can sweep them from the SLURM script."
7. *Infrastructure.* "The step-embedding builder gets killed on the login node. Write an `sbatch` script that runs it on a GPU node with appropriate memory and time limits."
8. *Result interpretation.* "The ensemble only gains $+0.001$ AUROC over the Longformer alone. What does this tell us about the marginal value of step embeddings, and what should we try next?"
9. *Compatibility.* "We got `CUDA error: no kernel image is available for execution on the device` on the B200 partition. Diagnose and fix the PyTorch–CUDA architecture mismatch."

Reproducibility checklist (Complete and submit as a standalone file with your project report)

* Please fill out the top part of the form first (first page).
* Please make sure the points in second page are addressed in your report submission
* Please copy this and replace the ☐ with a ✔ for the items that are addressed in your report/code submission
* Please complete this checklist, attach it to your final project report as the last page and then submit.

Your team member names (comma separated):

Lixing Lin, Youxuan Ma, Peng Chen

Your project title:

Reasoning Trace Topology as Calibration-Free Uncertainty Quantification for Large Language Models

AI Use Statement (see below for instructions and examples)

We used Anthropic's Claude (Sonnet 4.6 / Opus 4.6) as a coding and writing assistant during this project. It helped us with code review and debugging, as well as some statistical analysis. We also used it to review our report and incorporated its suggestions to polish and enhance our final draft. In addition, we used it as an idea sounding-board and discussed with it whether to drop or retain individual signal families and what ablations would be informative.

Author Contribution Statement (see below for instructions and examples)

Youxuan Ma handled all datasets ingestion and preparation, including trace generation, data preprocessing, integration, and feature engineering. Contributed to the architectural design and model development, and helped improve final model performance through several optimizations including multi-layer probes. Organized and maintained the code repo, and led the writing of the final report. Lixing Lin implemented the sequence-model baselines (DeBERTa, Longformer-base-4096), developed the step-embedding transformer, designed the weighted ensemble and hybrid meta-learner stacking pipeline, ran the main fine-tuning experiments on the Yale Bouchet HPC cluster, and optimized the training pipeline (pre-tokenization, focal loss, early stopping). Peng Chen designed the structure-aware feature pipeline, including a rule-based six-class behavior parser, 25

handcrafted descriptors (length, structural, meta), 5 semantic-recurrence features, 4 problem-trace alignment features, and 10 generation-uncertainty features from per-token logprob and entropy trajectories. Built the length-controlled evaluation framework. Designed and ran the hidden-state probing experiments, including the layer-position atlas sweep (8 layers, 4 positions, Qwen and Llama) and the multi-layer concatenation probe. Implemented the conformal-prediction wrapper with cross-family and leave-one-dataset-out coverage validation, and the per-dataset and cross-model transfer evaluation harness. Diagnosed and fixed a cartesian-product merge bug in the meta-learner that had inflated reported AUROC by over 0.10. Contributed to result analysis in the report.

Checklist (Make sure these are addressed in your submission and replace ☐ with a ✔ where applies)

Source Code Accessibility:

✔ Provide a link to the source code on github or submit the code repo to us separately if you have privacy concerns.

✔ Ensure the code is well-documented and easy to run

Computing Infrastructure:

✔ Mention the compute resources used (GPUs or API models you've used)

Dataset Description:

✔ Clearly describe and cite the datasets used, and any preprocessing/modifications done.

Hyperparameters and Tuning Process:

✔ Detail the hyperparameters used and the process for selecting them.
(Example: The model was fine-tuned using a batch size of 16, learning rate of 1e-5, and trained on 1000 steps with 100 steps of learning rate linear warmup with linear decay)

Experimental Results:

✔ Clearly define the evaluation metrics and statistical methods used in assessing the model.

✔ Present the main set of results in a table, chart or readable format

✔ Include comparisons with baseline models and state-of-the-art, where applicable.

AI Use

✔ Clear statement of how and where you have used AI in the project

✔ Specific parts where AI has helped

✔ Prompts that you used included as an auxiliary file or appendix in your report

Good Example 1: We used ChatGPT to help brainstorm project directions, clarify implementation details. We then used Claude Code to help write code and experiments. We used both ChatGPT and Claude Code to help plot bar charts for our raw results. All experimental design decisions, code changes, analysis, and final writing were reviewed and finalized by the team. We used ChatGPT to help with polishing the report write-up and help with grammar.

Good Example 2: We used Codex to assist with implementing preprocessing scripts for the summarization dataset, including tokenization, train/dev/test splitting, and formatting examples for model input. We also used Claude Code to help debug errors in the fine-tuning script and to refactor evaluation code for ROUGE scoring. All prompts, generated code, and edits were reviewed by the team before inclusion in the final repository. We did NOT use AI for the final project report writing extensively with exception of helping rephrasing a few sentences.

Bad example 1: We vibe-coded parts of the project using Claude Code.

Bad example 2: AI was used in Writing, Coding and Experiments.
(This doesn't say how exactly you used AI and in what specific parts)

Author Contribution Statement
✔ *Clear statement of what each member in your team has done.*

Good example 1: Alice Smith led dataset collection and preprocessing, including cleaning the raw data and creating the train, validation, and test splits. Bob Chen implemented the baseline and fine-tuned models, ran the main experiments, and organized the code repository. Carol Lee designed the evaluation framework, produced the result tables and figures, and wrote the analysis section of the report. All team members contributed to project planning, reviewed the final report, and approved the final submission.

Good example 2 (mix of grad and undergrad students): Bob Chen (UG) handled dataset preprocessing and exploratory analysis. Carol Lee (UG) implemented the model training pipeline and ran the hyperparameter tuning experiments. Alice Smith (Grad) conducted evaluation, created the result visualizations, and wrote the discussion of errors and limitations. Alice also conduced the literature review. All authors contributed to writing and editing the final report.

Bad example 1: All team members contributed equally to the project.
(This gives no information about each person's role or contribution.)

Bad example 2: Alice did coding, Bob did experiments, and Carol wrote the report.
(This is too general, you need to provide specific details on exactly who did what)